

Projet de programmation 2024-2025



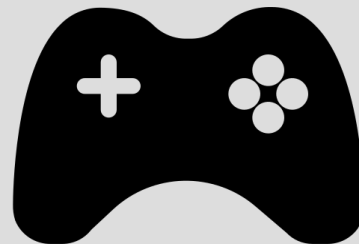
Objectif du projet

- Jeu dans un langage fonctionnel
- Utiliser les avantages d'OCaml pour éviter les effets de bord (modules, monads) et garantissant un code plus maintenable sur le long terme.



OCaml

+



Présentation du jeu

•Objectif du jeu :

- Parcourir le niveau en atteignant la porte et passer au niveau suivant
- Éviter/attaquer les ennemis.
- Sauter/utiliser le grappin de plateforme en plateforme.

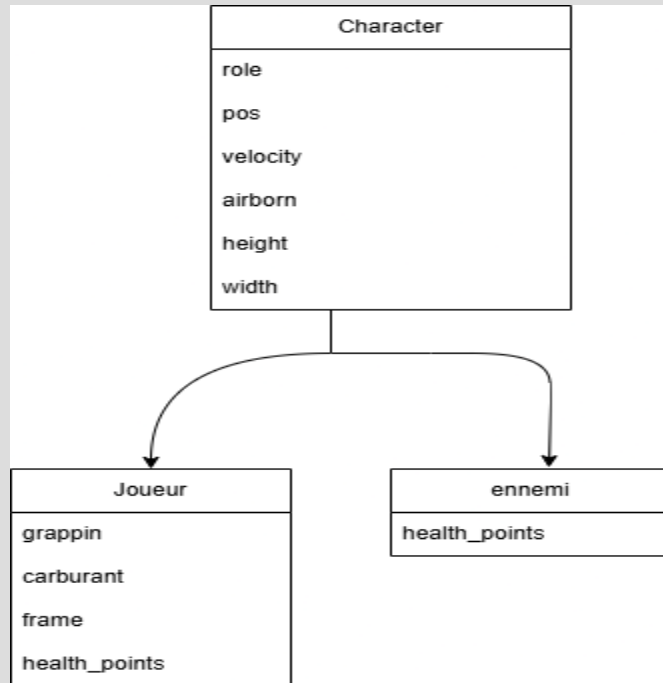
•Contrôles :

- Flèches : se déplacer de gauche à droite et sauter
- Espace : utiliser le grappin
- E : Attaquer

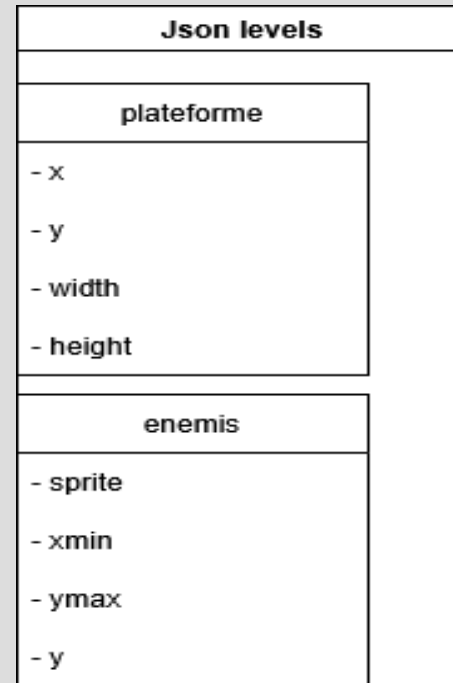


Organisation de l'architecture des entités

- Structure des entités



- Structure des niveaux



Structure du programme

- Setup du jeu
 - Chargement des textures
 - Création/Initialisation des entités
 - Entités : joueur ; ennemis ; plateformes
- Boucle du jeu
 - Actions des entités
 - Mise à jour des attributs
 - Affichage des textures

Génération/Création de niveaux

Les niveaux sont stockés dans un dossier level.

Les niveaux doivent avoir comme nom levelX avec X le numéro du niveau

```
{  
  "platforms": [  
    {  
      "x": 500,  
      "y": 400,  
      "width": 600,  
      "height": 30  
    },  
    {  
      "x": 0,  
      "y": 200,  
      "width": 400,  
      "height": 30  
    },  
    {  
      "x": 1600,  
      "y": 2000,  
      "width": 0,  
      "height": 2000  
    },  
    {  
      "x": 0,  
      "y": 2000,  
      "width": 0,  
      "height": 2000  
    }  
  ],  
}
```

```
"ennemis": [  
  {  
    "sprite": "../resources/knight.png",  
    "xmin": 1100.0,  
    "xmax": 1300.0,  
    "y": 150.0  
  },  
  {  
    "sprite": "../resources/knight.png",  
    "xmin": 600.0,  
    "xmax": 1000.0,  
    "y": 550.0  
  },  
  {  
    "sprite": "../resources/knight.png",  
    "xmin": 500.0,  
    "xmax": 850.0,  
    "y": 150.0  
  }  
]
```

Génération de niveaux : parsing des fichiers Json

```
let parse_json file =  
  let json = Basic.from_file file in  
  match json with  
  | `Assoc l -> if not (List.length l = 2) then failwith "wrong level format" else (  
    let p_list =  
      (let (_, pl) = List.hd l in match pl with  
      | `List plist -> List.map (fun p ->  
        match p with  
        | `Assoc jp -> List.map (fun champ -> snd champ) jp  
        | _ -> failwith "wrong json format"  
      ) plist  
      | _ -> failwith "wrong json format")  
    in let e_list =  
      (let (_, el) = List.nth l 1 in match el with  
      | `List elist -> List.map (fun e ->  
        match e with  
        | `Assoc je -> List.map (fun champ -> snd champ) je  
        | _ -> failwith "wrong json format"  
      ) elist  
      | _ -> failwith "wrong json format")  
    in (p_list, e_list)  
  )  
  | _ -> failwith "not a json object"
```

Style fonctionnel : Utilisation des monads

.Étapes d'implémentation

```
let joueur1 = { x = 0.0; y = 0.0; healthpoint = 100; atk = 10; nom = "eren" }
```

```
let joueur2 = deplacer_joueur joueur1 5.0 3.0  
let joueur3 = infliger_degats joueur2 20  
let joueur4 = changer_nom joueur3 "Lancelot"
```



```
let programme =  
  let* () = set_position 10.0 15.0 in  
  let* () = set_healthpoint 100 in  
  let* () = set_nom "Arthur" in  
  return ()
```



.Utilisation de la monade d'état :

CharacterMonad :

```
type 'a t = character -> 'a * character
```

fonctions usuelles de la monade d'état :

-return

-bind

-let*

-get

fonctions spécifiques à CharacterMonad :

-create_character

-deplacement

-patrol

-set_health

-add_carburant

...

Possible v2 : fonctionnalités bonus

- Jeu à plusieurs (local ou réseau)



- Ajouter des items que le joueur pourra collecter



- Ajouter un jetpack au joueur avec, comme pour le grappin, une barre d'énergie et un type de déplacement spécifique



- Améliorer l'algorithme des ennemis